

# 分布式训练与加速

随着大语言模型参数规模从亿级走向万亿级，单机训练已完全无法满足算力与显存需求。分布式训练是现代大模型工程的核心技术，涉及数据并行、模型并行、显存优化、混合精度、通信优化等多个维度。本文系统介绍分布式训练的关键技术与主流框架。

## 分布式训练与加速

随着大语言模型参数规模从亿级走向万亿级，单机训练已完全无法满足算力与显存需求。分布式训练是现代大模型工程的核心技术，涉及数据并行、模型并行、显存优化、混合精度、通信优化等多个维度。本文系统介绍分布式训练的关键技术与主流框架。

- - -

### 一、单机训练瓶颈分析

#### 1.1 显存瓶颈

以 175B 参数模型为例，仅模型参数（FP16）就需 350 GB，加上梯度与优化器状态，单机训练所需显存远超单卡 80 GB 上限。显存占用构成（使用 Adam FP16 混合精度）：

组成 大小（相对参数量 P）

模型参数（FP16） 2P

梯度（FP16） 2P

优化器状态（FP32 动量 + 方差 + 主参数） 12P

激活值（依赖 batch 与序列长度） 可观

总显存约 16P 字节，175B 模型需约 2.8 TB 显存。

#### 1.2 算力瓶颈

单卡 FLOPS 有限，即使 8 卡 A100 也难以在合理时间内完成万亿参数训练。算力需求  $\approx \sqrt{N} \times P$ （Chinchilla 缩放定律），N 为 token 数，P 为参数量。

#### 1.3 通信瓶颈

多卡协同需要交换梯度或激活，PCIe/NVLink/InfiniBand 带宽成为瓶颈。NVLink 30 GB/s，InfiniBand 400 Gbps，远低于显存带宽（约 2 TB/s）。

- - -

## 二、数据并行 (Data Parallelism)

### 2.1 基本原理

每张 GPU 持有完整模型副本，分别处理不同数据子集，前向后通过 AllReduce 同步梯度。

同步 SGD 流程：

1. 各 GPU 分别前向、反向计算梯度  $g_i$
2. AllReduce 求平均： $g = \frac{1}{N} \sum g_i$
3. 各 GPU 用相同梯度更新参数

### 2.2 通信开销

AllReduce 通信量与参数量成正比，约为  $2P \cdot \text{dtype}$  字节。算法带宽利用率高，通信时间随 GPU 数对数增长。

### 2.3 局限

每张卡需容纳完整模型与优化器状态，无法训练超过单卡显存的模型，因此催生了 ZeRO、FSDP 等分

---

## 三、模型并行

### 3.1 张量并行 (Tensor Parallelism, TP)

将单个层 (如 Linear、Attention) 的权重矩阵按行或列切分到多张卡上，每张卡计算部分结果，AllReduce/All-Gather 拼接。

以  $Y = XW$  为例 (列切分)：

$$W = [W_1, W_2, \dots, W_n], \quad Y = [XW_1, XW_2, \dots, XW_n]$$

每张卡只需存储  $W_i$ ，前向无需通信，反向需 AllReduce 梯度。

特点：通信频繁，建议只在节点内 (NVLink 域内) 使用，典型规模 8 卡。

### 3.2 流水线并行 (Pipeline Parallelism, PP)

将模型按层切分到多张卡，输入按微批次（micro - batch）流过各阶段。

朴素流水线会产生 " 气泡 " （bubble），GPU 空闲。改进方法：

- GPipe：将一个 batch 切成 M 个微批次，并行填充流水线
- 1F1B（PipeDream）：一个前向紧跟一个反向，减少激活显存
- Interleaved 1F1B：每个设备负责多个虚拟阶段，进一步缩小气泡

气泡占比约为  $\frac{P-1}{M}$ （P 为阶段数，M 为微批次数），M 越大气泡越小。

3.3 三种并行对比

维度 数据并行（DP） 张量并行（TP） 流水线并行（PP）

切分对象 数据 层内权重 层间分 stage

通信频率 每步一次 AllReduce 每层前后向都通信 stage 间传递激活

通信带宽需求 中 高 低

推荐位置 跨节点 节点内 跨节点

显存节省 无 显著 显著

- - -

四、ZeRO 优化（ZeRO - 1 / 2 / 3）

ZeRO（Zero Redundancy Optimizer）通过分片消除 DP 中的显存冗余。

4.1 三级优化

阶段 分片对象 显存节省 通信开销（相对 DP）

ZeRO - 1 优化器状态（ $1/2 P$ ）  $4\times$  与 DP 相同

ZeRO - 2 优化器状态 + 梯度（ $1/4 P$ ）  $8\times$  略增（Reduce - Scatter）

ZeRO - 3 优化器状态 + 梯度 + 参数（ $1/6 P$ ）  $N\times$  显著增加（前向反向都需 AllReduce）

4.2 显存公式

设 N 为 GPU 数，ZeRO - 3 显存约为  $\frac{1/6 P}{N} + 2 P \times \text{activation}$

4.3 选择策略

- 模型可装入单卡：使用 DP 或 ZeRO - 1
- 模型装不下但激活可控：ZeRO - 2
- 模型远大于单卡容量：ZeRO - 3 (结合 CPU / NVMe offload)

- - -

## 五、FSDP (Fully Sharded Data Parallel)

PyTorch 原生实现的 ZeRO - 3 等价方案，自 1.11 起逐步成熟。

### 5.1 工作机制

1. 前向前：All - Gather 拉取完整参数 → 计算后丢弃非本分片
2. 反向前：再次 All - Gather 拉取参数 → 计算梯度
3. 梯度同步：Reduce - Scatter 各卡只保留自己分片的梯度
4. 优化器更新：仅更新本分片参数

### 5.2 与 ZeRO - 3 的关系

FSDP 在概念上等价于 ZeRO - 3，但深度集成于 PyTorch 生态，与 `torch.distributed`、`torch.compile`、混合精度协同良好。

```
from torch.distributed.fsdp import FullyShardedDataParallel
from torch.distributed.fsdp import ShardingStrategy
import torch.distributed as dist
```

```
dist.init_process_group("nccl")
```

```
model = build_large_model().cuda()
```

```
mppolicy = MixedPrecision(param_dtype=torch.bfloat16,
    reduced_type=torch.bfloat16,
    buffer_dtype=torch.bfloat16)
```

```
model = FSDP(model,
    sharding_strategy=ShardingStrategy.FULLSHARD,
    mixed_precision=mppolicy,
    device_id=torch.cuda.current_device())
```

```
optimizer = torch.optim.AdamW(model.parameters()),
```

后续训练循环与单机一致

- - -

## 六、DeepSpeed 与 Megatron-LM

### 6.1 DeepSpeed

微软开源的训练优化库，核心能力：

- ZeRO-1/2/3：显存分片
- ZeRO-Offload：将优化器状态卸载到 CPU
- ZeRO-Infinity：进一步卸载到 NVMe，支持超大规模
- MoE 训练：支持专家并行
- 1-bit Adam / 0/1 Adam：压缩梯度通信

通过 JSON 配置文件即可启用：

```
{
  "zerooptimization": {
    "stage": 3,
    "offloadoptimizer": { "device": "cpu" },
    "offloadparam": { "device": "none" }
  },
  "bf16": { "enabled": true },
  "gradientaccumulationsteps": 8,
  "gradientclipping": 1.0
}
```

### 6.2 Megatron-LM

NVIDIA 开源的大模型训练参考实现，重点在并行策略：

- 张量并行：行列切分 Linear 与 Attention
- 流水线并行：支持 1F1B 与 Interleaved

- 序列并行：将序列维度切分，降低 `Activation` 显存
- 3D 并行：`DP × TP × PP` 组合，按拓扑感知分配

### 6.3 二者对比

维度 `DeepSpeed Megatron-LM`

重点 显存与通信优化 并行策略与拓扑

易用性 配置驱动，门槛低 需改模型代码，门槛高

典型场景 中小团队微调 千卡预训练大模型

生态 `HuggingFace` 集成好 `NVIDIA` 生态原生

实践中常将二者结合：`Megatron` 负责并行拓扑，`DeepSpeed` 负责 `ZeRO` 与 `offload`

- - -

## 七、混合精度训练

### 7.1 三种数值格式

格式 指数位 尾数位 动态范围 精度

`FP32` 8 23 高 高

`FP16` 5 10 低 中

`BF16` 8 7 高（同 `FP32`） 低

`FP8` (`E4M3 / E5M2`) 4 / 5 3 / 2 中 / 高 极低

### 7.2 自动混合精度 (AMP)

`FP16` 训练需配合 `loss scaling` 防止小梯度下溢：

```
forward (FP16) → loss → loss * scale → backward (FP16)
→ unscale grads → optimizer step (FP32 master weights)
```

`BF16` 动态范围与 `FP32` 相同，无需 `loss scaling`，是当前大模型训练首选。`FP8` 支持，可进一步提速 1.5 - 2×。

```
from torch.cuda.amp import autocast, GradScaler
```

```

scaler = GradScaler() # FP16 需要
for x, y in dataloader:
    optimizer.zero_grad()
    with autocast(dtype=torch.bfloat16): # BF16 无需 scaler
        out = model(x)
        loss = criterion(out, y)
    if use_fp16:
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
    else:
        loss.backward()
        optimizer.step()

- - -

```

## 八、梯度累积与梯度检查点

### 8.1 梯度累积

当显存不足以容纳大 `batch` 时，将一个逻辑 `batch` 拆成多个微批次累加梯度，达到目标 `batch` 后再更新参数：

$$g = \frac{1}{M} \sum_{m=1}^M g_m, \quad \theta \leftarrow \theta - \eta g$$

等效 `batch size = microbatch × accumulation steps ×`  
统计仍按微批次计算，需用 `SyncBN` 或避免 `BN`。

### 8.2 梯度检查点 (Gradient Checkpointing)

以时间换空间：前向时只保存部分层激活，反向时重新计算被丢弃的激活。显存从  $O(L)$  降至  $O(\sqrt{L})$ ，计算量增加约 33%。

```

from torch.utils.checkpoint import checkpointsequential

```

`Transformer` 层使用检查点

```

layers = nn.ModuleList([TransformerBlock() for i in range(N)])
hidden = checkpointsequential(layers, numchunks=8)

```

- - -

九、分布式训练框架对比

框架 后端 易用性 主要特点

DDP (PyTorch 原生) NCCL / Gloo 高 数据并行首选

FSDP NCCL 中高 ZeRO - 3 等价

DeepSpeed NCCL 中 ZeRO / offload 丰富

Megatron-LM NCCL 低 3D 并行强

Accelerate 多后端 高 统一抽象层

Ray Train 多后端 中 与 Ray 生态集成

TorchTitan NCCL 中 PyTorch 官方大模型参考

9.1 选型建议

- 微调 7B - 70B : FSDP 或 DeepSpeed ZeRO - 2 / 3
- 从头预训练 100B+ : Megatron-LM + DeepSpeed (或 TorchTitan)
- 多机多卡小模型 : DDP
- 快速原型 : HuggingFace Accelerate

- - -

十、训练监控与故障恢复

10.1 关键监控指标

类别 指标 含义

性能 throughput (tokens / s) 训练速度

显存 peak memory 峰值显存占用

通信 NCCL bandwidth 通信带宽利用率

数值 loss / grad norm 训练健康度

稳定性 MFU (Model FLOPs Utilization) 算力利用率

一致性 gradient / param norm 检测异常

10.2 常见故障与处理

- `Loss spike`：梯度爆炸，降低学习率或启用梯度裁剪
- `NaN / Inf`：检查 `BF16 / FP16` 数值溢出，启用 `loss scaling`
- `NCCL 超时`：网络抖动，调整 `NCCLTIMEOUT`，检查 `InfiniBand`
- 节点掉线：自动重训练（见下）

### 10.3 故障恢复机制

1. 定期 `Checkpoint`：保存模型 / 优化器 / 调度器 / 数据加载器状态
2. 异步 `Checkpoint`：避免同步保存阻塞训练（如 `DeepSpeed` 异步保存）
3. 弹性训练：`TorchElastic` 支持节点动态加入 / 退出，自动重新组网
4. 冗余保存：本地 + 共享存储（`S3 / NFS`）双写

`TorchRun` 弹性训练示例

```
torchrun \
--nnodes=2-4 \
--nprocpernode=8 \
--rdzvbackend=c10d \
--rdzvendpoint=leader:29500 \
train.py --config config.yaml
```

### 10.4 `Checkpoint` 保存（`FSDP` 示例）

```
from torch.distributed.fsdp import FullyShardedDataParallel
import os

def savecheckpoint(model, optimizer, step, rank,
if rank == 0:
os.makedirs(path, exist_ok=True)
torch.distributed.barrier()
# FSDP 需要 sharded state dict
state = {
"model": model.state_dict(),
"optimizer": optimizer.state_dict(),
"step": step,
}
```

```
# 各 rank 保存自己分片
```

```
torch.save(state, f"{path}/rank{rank}.pt")
```

加载时需先初始化 FSDP，再 loadstate\_dict

- - -

## 十一、关键词

分布式训练、数据并行、模型并行、ZeRO、DeepSpeed、Megatron、混合精度